CLD-105 · Orchestration de conteneurs avancé (Kubernetes/Git)

Compte rendu – TP4

Configuration, santé et stockage

Élément	Valeur
Étudiant	Qassim Messili
Formation	Mastère (Bac+5) – IPI Lyon
Date	27 octobre 2026
Cluster	kubeadm : `k8s-cp` (10.2.100.225) et `k8s-w1` (10.2.100.219)
Application	example-voting-app, namespace `vote`
Dossier de travail	`~/tp4` (copie de `~/tp3`, gardé intact)

Sommaire

Si le sommaire est vide : clic droit → « Mettre à jour les champs » (ou F9 dans Word).

Introduction

À la fin du TP3, mon application de vote fonctionnait, mais elle n'aurait pas tenu en production. Le TP4 corrige ses quatre gros défauts, dans l'ordre où une équipe le ferait :

Défaut au TP3	Correction au TP4
Options et mot de passe écrits en dur dans les fichiers	**ConfigMap** et **Secret**
Kubernetes ne sait pas si l'application va bien	**Sondes de santé** (startup, readiness, liveness)
Aucune limite de CPU ou de mémoire	**Requests/limits**, **LimitRange** et **ResourceQuota**
La base de données perd tout à chaque redémarrage	**PVC**, **StorageClass** puis **StatefulSet**

J'ai terminé par un **PodDisruptionBudget**, qui protège l'application pendant les maintenances. Avant de commencer, j'ai copié le dossier du TP3 pour garder une trace de l'état de départ : au jour 3, ces changements deviendront un historique Git.

VM 1 – k8s-cp

```
cp -r ~/tp3 ~/tp4
cd ~/tp4
kubectl get nodes
kubectl -n vote get pods
```

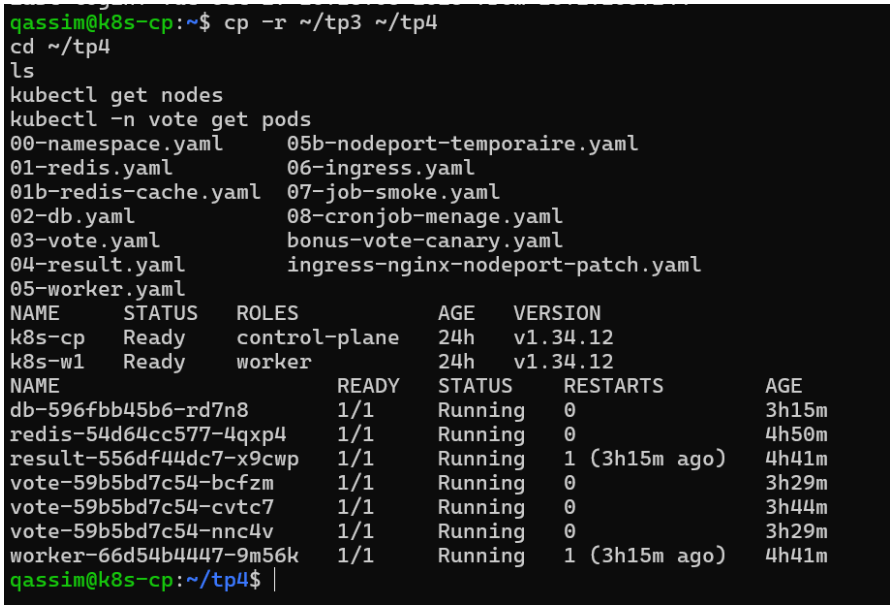


Figure 1 – Point de départ : les fichiers du TP3 et l'application en marche

Partie 1 – ConfigMap et Secret

Sortir les options de vote dans une ConfigMap

Une **ConfigMap** est un objet qui contient de la configuration (des paires clé/valeur), séparée de la description de l'application. J'y ai mis les deux options de vote :

10-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: vote-config
  namespace: vote
data:
  OPTION_A: "Montagne"
  OPTION_B: "Plage"
```

Dans le Deployment vote, j'ai remplacé le bloc `env` (les options écrites en dur) par un `envFrom`, qui charge d'un coup toutes les clés de la ConfigMap comme variables d'environnement :

03-vote.yaml (extrait)

```
        envFrom:
        - configMapRef:
            name: vote-config
```

VM 1

```
kubectl apply -f 10-configmap.yaml -f 03-vote.yaml
kubectl -n vote rollout status deployment/vote
```

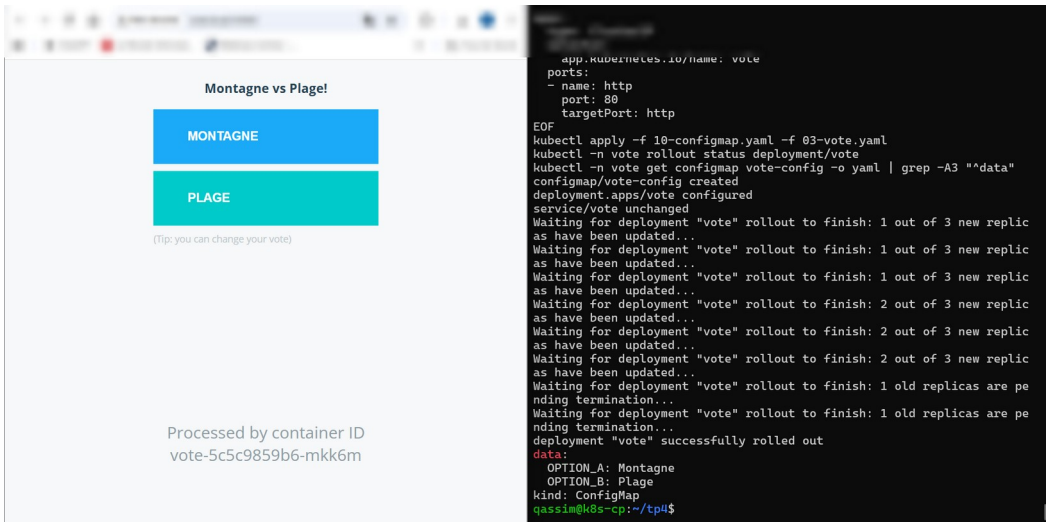


Figure 2 – Le rollout de vote et la page qui affiche Montagne vs Plage

Le template du Deployment a changé, donc Kubernetes a fait un **rollout** pod par pod, et la page affiche bien les nouvelles options.

Modifier la ConfigMap seule

J'ai ensuite changé une valeur dans la ConfigMap (Montagne → Campagne) et je l'ai appliquée seule. Résultat : la ConfigMap contenait bien « Campagne », mais **la page affichait toujours « Montagne »**, et les pods vote n'avaient pas redémarré. J'ai alors forcé la prise en compte :

VM 1

```
sed -i 's/OPTION_A: "Montagne"/OPTION_A: "Campagne"/' 10-configmap.yaml
kubectl apply -f 10-configmap.yaml          # la page affiche toujours Montagne
kubectl -n vote rollout restart deployment/vote
```

Question 1 – Les pods affichent-ils la nouvelle valeur ? Non. Les variables d'environnement sont lues **une seule fois, au démarrage** du conteneur. Modifier la ConfigMap ne touche pas au template du Deployment : aucun rollout n'est lancé, et les pods gardent l'ancienne valeur. Deux façons de propager le changement :

• **impérative** : `kubectl rollout restart deployment/vote` (Kubernetes ajoute une date dans le template, ce qui relance les pods) ;

• **déclarative** : faire changer le template en même temps que la ConfigMap, par exemple en la renommant (`vote-config-v2`) et en mettant à jour la référence, ou en ajoutant une annotation qui contient une empreinte (hash) du contenu. C'est ce que fera automatiquement `configMapGenerator` de Kustomize au jour 3.

Le mot de passe de la base dans un Secret

Un **Secret** ressemble à une ConfigMap, mais il est fait pour les données sensibles. Je l'ai généré avec kubectl, sans le créer directement (option `--dry-run`), pour obtenir un fichier :

VM 1

```
kubectl -n vote create secret generic db-credentials \
  --from-literal=POSTGRES_USER=postgres \
  --from-literal=POSTGRES_PASSWORD=postgres \
  --dry-run=client -o yaml > 11-secret.yaml
echo cG9zdGdyZXM= | base64 -d      # affiche : postgres
```

Dans le fichier, les valeurs apparaissent sous la forme `cG9zdGdyZXM=`. Ce n'est **pas du chiffrement**, juste un encodage en base64 : n'importe qui peut le décoder en une commande. La base db lit maintenant ses identifiants dans ce Secret :

02-db.yaml (extrait)

```
        envFrom:
        - secretRef:
            name: db-credentials
```

```
    app.kubernetes.io/part-of: voting-app
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: db
  ports:
  - name: postgres
    port: 5432
    targetPort: postgres
EOF
kubectl apply -f 11-secret.yaml -f 02-db.yaml
kubectl -n vote rollout status deployment/db
kubectl -n vote get pods
kubectl -n vote exec deploy/db -- printenv POSTGRES_USER
secret/db-credentials created
deployment.apps/db configured
service/db unchanged
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 of 1 updated replicas
 are available...
deployment "db" successfully rolled out
NAME                      READY   STATUS    RESTARTS        AGE
db-55d4dd76c7-7hhh7       1/1     Running   0               1s
redis-54d64cc577-4qxp4    1/1     Running   0               4h58m
result-556df44dc7-x9cwp   0/1     Error     1 (3h23m ago)   4h49m
vote-69f6d9d74f-gkxll     1/1     Running   0               48s
vote-69f6d9d74f-kv9hz     1/1     Running   0               50s
vote-69f6d9d74f-r649v     1/1     Running   0               53s
worker-66d54b4447-9m56k   1/1     Running   2 (2s ago)      4h49m
postgres
qassim@k8s-cp:~/tp4$
```

Figure 3 – Le Secret est créé et la base lit bien POSTGRES_USER depuis le Secret

`printenv POSTGRES_USER` renvoie bien `postgres`. Au moment de la capture, result et worker sont en erreur : ils venaient de perdre la base, remplacée par le rollout. Ils sont revenus seuls quelques secondes plus tard :

```
qassim@k8s-cp:~/tp4$ sleep 20
kubectl -n vote get pods
NAME                      READY   STATUS    RESTARTS      AGE
db-55d4dd76c7-7hhh7       1/1     Running   0             78s
redis-54d64cc577-4qxp4    1/1     Running   0             4h59m
result-556df44dc7-x9cwp   1/1     Running   2 (79s ago)   4h50m
vote-69f6d9d74f-gkxll     1/1     Running   0             2m5s
vote-69f6d9d74f-kv9hz     1/1     Running   0             2m7s
vote-69f6d9d74f-r649v     1/1     Running   0             2m10s
worker-66d54b4447-9m56k   1/1     Running   2 (79s ago)   4h50m
qassim@k8s-cp:~/tp4$
```

Figure 4 – Tous les pods sont revenus en Running

Question 2 – Quel intérêt d'un Secret si le mot de passe est imposé ?

• **Séparation des droits** : avec RBAC, on peut laisser quelqu'un lire les Deployments sans lui donner accès aux Secrets.

• **Meilleure protection** : les Secrets peuvent être chiffrés dans etcd et sont montés en mémoire quand on les utilise en volume.

• **La bonne habitude** : le jour où le mot de passe devient libre, on change le Secret, sans toucher au Deployment.

> Pour pouvoir choisir un vrai mot de passe, il faudrait modifier le code du **worker** (`Program.cs`) et de **result** (`server.js`), où la connexion à la base est écrite en dur, pour qu'ils lisent leurs identifiants dans des variables d'environnement, puis leur donner le même Secret.

Partie 2 – Les sondes de santé

Jusqu'ici, Kubernetes savait seulement si un conteneur tournait ou s'il avait planté. Avec les **sondes**, il vérifie régulièrement que l'application répond vraiment. Il en existe trois :

Sonde	Question posée	Si elle échoue
startup	« As-tu fini de démarrer ? »	On attend. Les deux autres sondes ne commencent qu'après sa réussite.
readiness	« Es-tu prêt à recevoir des visiteurs ? »	Le pod est **retiré du Service** (plus de trafic), mais il reste en vie.
liveness	« Es-tu toujours vivant ? »	Le kubelet **redémarre** le conteneur.

Les sondes de chaque composant

Pour **vote** et **result** (des sites web), les trois sondes font une requête HTTP sur `/` :

03-vote.yaml et 04-result.yaml (extrait)

```
        startupProbe:
          httpGet: { path: /, port: http }
          periodSeconds: 2
          failureThreshold: 30      # jusqu'à 60 s pour démarrer
        readinessProbe:
          httpGet: { path: /, port: http }
          periodSeconds: 5
          failureThreshold: 3
        livenessProbe:
          httpGet: { path: /, port: http }
          periodSeconds: 10
          timeoutSeconds: 2
          failureThreshold: 3
```

redis n'est pas un site web : sa readiness vérifie que le port 6379 accepte une connexion (`tcpSocket`), et sa liveness lance la commande `redis-cli ping` dans le conteneur (`exec`). Pour **db**, les deux sondes lancent `pg_isready -U postgres`, l'outil officiel de PostgreSQL pour savoir si la base répond. La liveness de db tolère 5 échecs, car une base peut être lente un court instant.

01-redis.yaml (extrait)

```
        readinessProbe:
          tcpSocket: { port: redis }
          periodSeconds: 5
        livenessProbe:
          exec: { command: ["redis-cli", "ping"] }
          periodSeconds: 10
```

02-db.yaml (extrait)

```
        readinessProbe:
          exec: { command: ["pg_isready", "-U", "postgres"] }
          periodSeconds: 5
        livenessProbe:
          exec: { command: ["pg_isready", "-U", "postgres"] }
          periodSeconds: 10
```

```
          failureThreshold: 5
```

Le **worker** reste sans sonde. Je l'ai expliqué dans un commentaire du fichier :

05-worker.yaml (extrait)

```
# Pas de sonde : il n'écoute sur aucun port (httpGet et tcpSocket sont
# impossibles) et son image ne contient pas d'outil de diagnostic (exec inutile).
# Pour en avoir une, il faudrait modifier l'application pour qu'elle expose un
# endpoint HTTP /healthz, ou qu'elle écrive régulièrement un fichier témoin
# (ex. /tmp/alive) dont une sonde exec vérifierait la date de mise à jour.
```

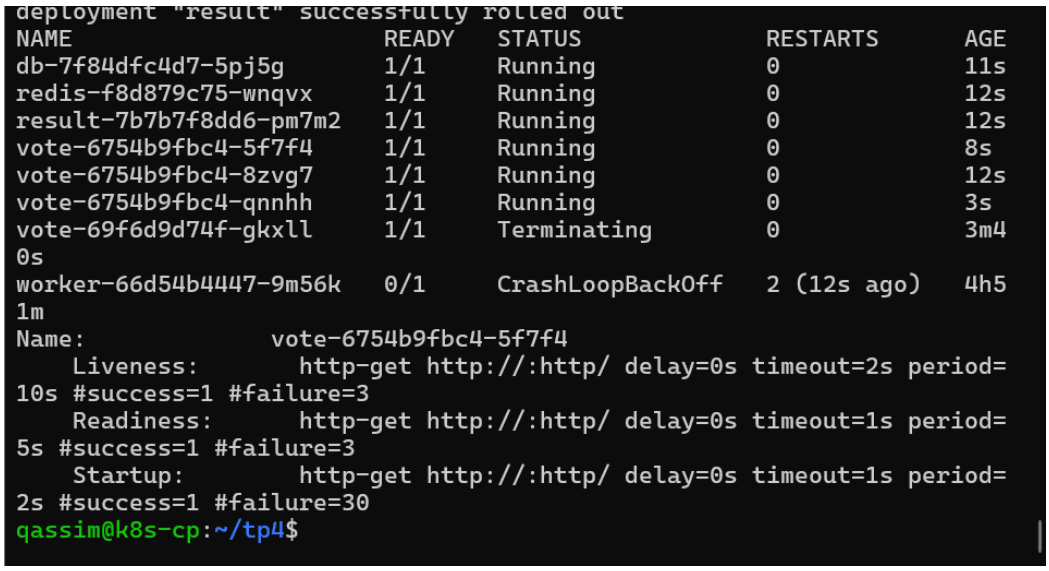


Figure 5 – Les trois sondes visibles dans kubectl describe d'un pod vote

`kubectl describe` montre bien les trois sondes avec leurs réglages. Le worker apparaît en `CrashLoopBackOff` : la base venait d'être recréée et il avait perdu sa connexion. Il est revenu seul :

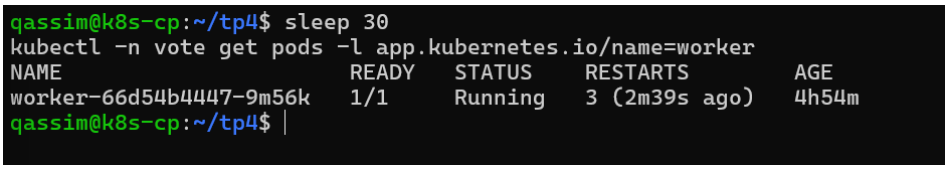


Figure 6 – Le worker revenu en Running sans intervention

Un rollout avec les sondes

J'ai relancé un rollout de vote (nouvelle option « Mer ») en surveillant les pods dans un second terminal :

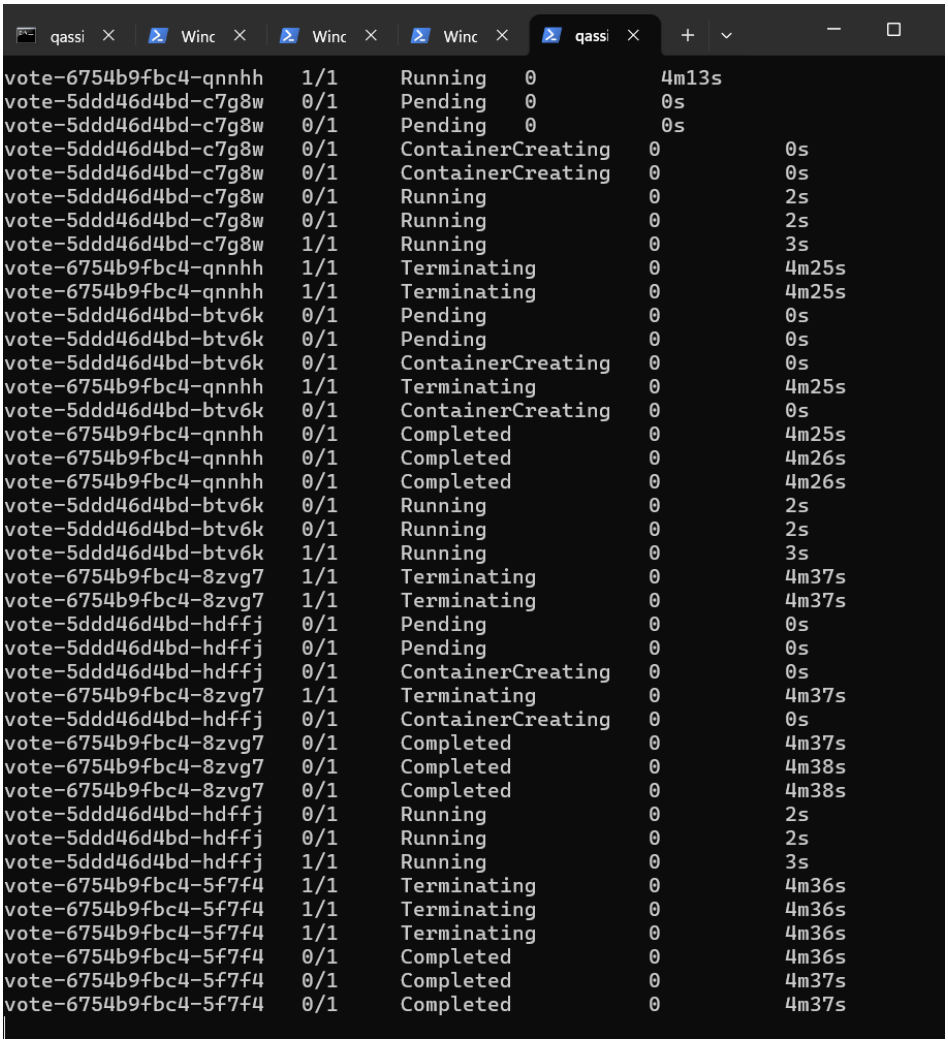


Figure 7 – Chaque nouveau pod passe par 0/1 avant 1/1, puis seulement un ancien s'arrête

```
qassim@k8s-cp:~/tp4$ cd ~/tp4
sed -i 's/OPTION_B: "Plage"/OPTION_B: "Mer"/' 10-configmap.yaml
kubectl apply -f 10-configmap.yaml
kubectl -n vote rollout restart deployment/vote
kubectl -n vote rollout status deployment/vote
configmap/vote-config configured
deployment.apps/vote restarted
Waiting for deployment "vote" rollout to finish: 0 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 old replicas are pe
nding termination...
Waiting for deployment "vote" rollout to finish: 1 old replicas are pe
nding termination...
deployment "vote" successfully rolled out
qassim@k8s-cp:~/tp4$
```

Figure 8 – Le rollout dans le premier terminal

Les nouveaux pods ne passent pas tout de suite à 1/1. Ils restent un court instant en `0/1 Running` : le conteneur tourne, mais la readiness n'a pas encore reçu de réponse. Pendant ce temps, aucun trafic ne leur est envoyé, et aucun ancien pod n'est supprimé. Ce n'est qu'au passage à `1/1` qu'un ancien pod s'arrête. Sans sonde, un pod serait annoncé prêt dès le lancement du programme, avant même de pouvoir répondre, et les premiers visiteurs recevraient des erreurs.

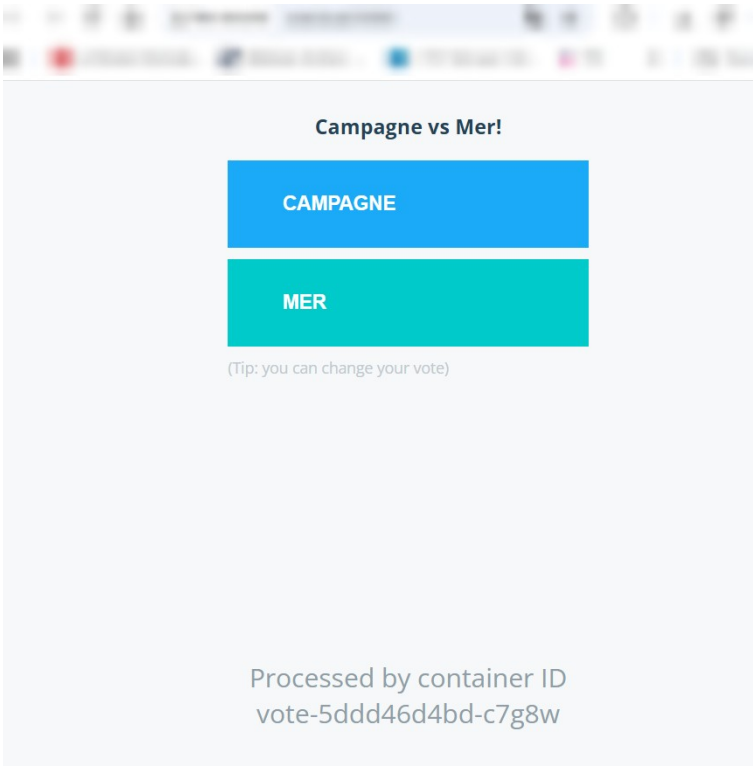


Figure 9 – La page affiche les nouvelles options : Campagne vs Mer

Expérience : une liveness mal réglée

J'ai volontairement pointé la liveness de vote vers une page qui n'existe pas (`/nexistepas`) :

VM 1

```
kubectl -n vote patch deployment vote --type=json \
  -p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/
httpGet/path","value":"/nexistepas"}]'
```

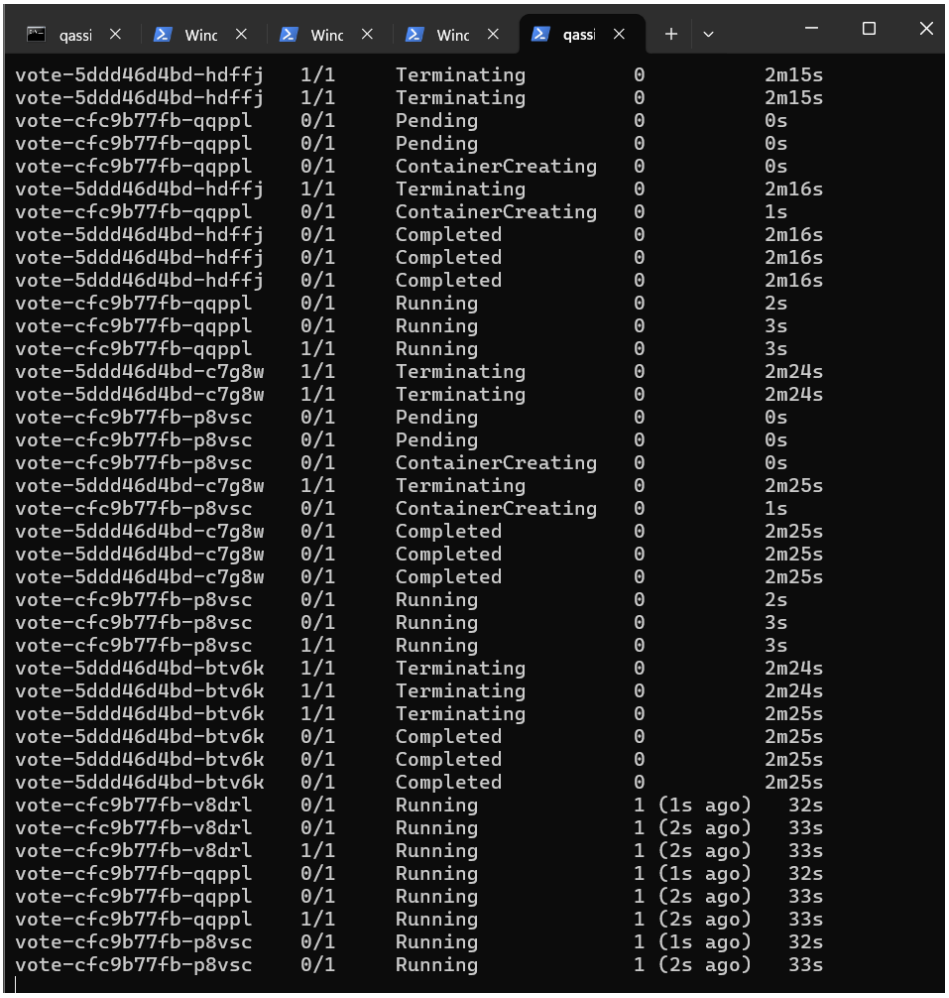


Figure 10 – Au bout de 30 secondes, chaque pod vote redémarre

Environ 30 secondes après leur démarrage, les trois pods affichent `RESTARTS 1`, puis le compteur augmente régulièrement. J'ai ensuite remis le bon chemin en réappliquant simplement le fichier :

```
qassim@k8s-cp:~/tp4$ kubectl apply -f 03-vote.yaml
kubectl -n vote rollout status deployment/vote
kubectl -n vote get pods -l app.kubernetes.io/name=vote
deployment.apps/vote configured
service/vote unchanged
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 2 out of 3 new replic
as have been updated...
Waiting for deployment "vote" rollout to finish: 1 old replicas are pe
nding termination...
Waiting for deployment "vote" rollout to finish: 1 old replicas are pe
nding termination...
deployment "vote" successfully rolled out
NAME                    READY   STATUS        RESTARTS      AGE
vote-5ddd46d4bd-8dxzn   1/1     Running       0             6s
vote-5ddd46d4bd-hksd9   1/1     Running       0             9s
vote-5ddd46d4bd-xd49f   1/1     Running       0             3s
vote-cfc9b77fb-v8drl    1/1     Terminating   3 (14s ago)   105s
qassim@k8s-cp:~/tp4$
```

Figure 11 – Retour au bon réglage : 3 pods neufs sans redémarrage, l'ancien en avait 3

> **Question 3 – Qu'a-t-on observé ?** La page `/nexistepas` renvoie une erreur 404, donc la liveness échoue. Après 3 échecs à 10 secondes d'intervalle (30 s), le kubelet **tue et redémarre** le conteneur : la colonne RESTARTS augmente d'environ 1 toutes les 30 s (le dernier pod en était à 3 au bout de 105 s). Le STATUS reste Running, avec un passage en 0/1 après chaque redémarrage, et finit en `CrashLoopBackOff` à force d'échecs.
>
> **Pourquoi la readiness n'aurait pas fait ça ?** Une readiness en échec **ne redémarre jamais** le conteneur : elle retire seulement le pod du Service. Le pod serait resté vivant, sans trafic. La liveness est donc bien plus dangereuse quand elle est mal réglée : elle fait redémarrer en boucle une application qui marche. On la choisit simple, et plus tolérante que la readiness.

Détail intéressant : les nouveaux pods portent le même préfixe qu'avant l'expérience (`vote-5ddd46d4bd`). Le template étant redevenu identique, Kubernetes a réutilisé l'ancien ReplicaSet.

Partie 3 – Ressources, LimitRange et ResourceQuota

Requests et limits

Pour chaque conteneur, on déclare deux valeurs :

- les **requests** : ce que le conteneur est **sûr d'obtenir**. Le scheduler s'en sert pour choisir un nœud qui a assez de place ;
- les **limits** : le **maximum** que le conteneur a le droit d'utiliser.

03-vote.yaml (extrait)

```
        resources:
          requests: { cpu: 50m, memory: 64Mi }
          limits:   { cpu: 250m, memory: 128Mi }
```

Composant	Requests (CPU / mémoire)	Limits (CPU / mémoire)
vote, result	50m / 64Mi	250m / 128Mi
redis	50m / 32Mi	250m / 128Mi
worker	50m / 96Mi	500m / 256Mi
db	100m / 128Mi	500m / 256Mi

`50m` veut dire 50 « millicœurs », soit 5 % d'un processeur.

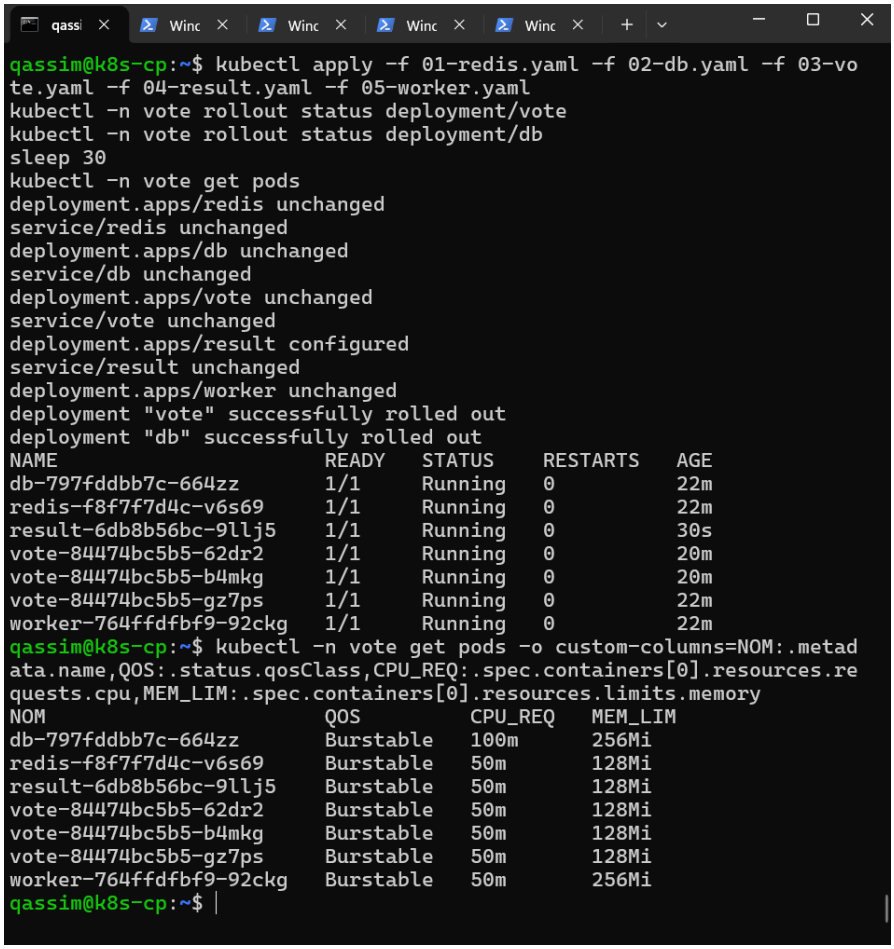


Figure 12 – Ressources appliquées et classe QoS de chaque pod

J'avais d'abord créé ces fichiers dans le mauvais dossier après une coupure SSH (`~` au lieu de `~/tp4`). Je les ai simplement déplacés au bon endroit.

> **Les classes QoS.** Tous mes pods sont **Burstable**, car leurs requests sont plus petites que leurs limits. Kubernetes range les pods en trois classes, qui décident de l'ordre d'expulsion quand un nœud manque de mémoire :
>
> • **Guaranteed** (requests = limits) : expulsés en dernier ;
>
> • **Burstable** (requests < limits) : notre cas ;
>
> • **BestEffort** (rien de déclaré, comme au TP3) : expulsés en premier.

LimitRange et ResourceQuota

Ces deux objets encadrent le namespace entier :

12-limitrange.yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: vote
spec:
  limits:
  - type: Container
    default:        { cpu: 500m, memory: 256Mi }
    defaultRequest: { cpu: 50m,  memory: 64Mi }
    max:            { cpu: "1",  memory: 1Gi }
    min:            { cpu: 10m,  memory: 16Mi }
```

13-resourcequota.yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: vote-quota
  namespace: vote
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.memory: 4Gi
    pods: "15"
    persistentvolumeclaims: "4"
```

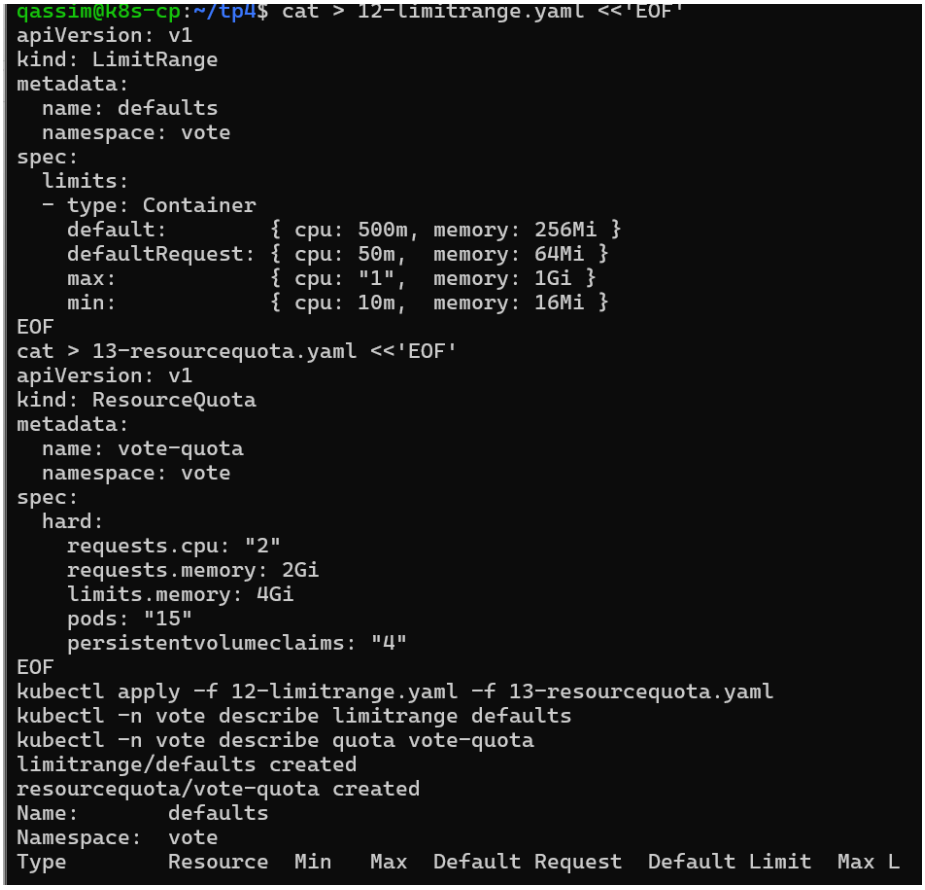


Figure 13 – Création du LimitRange et du ResourceQuota

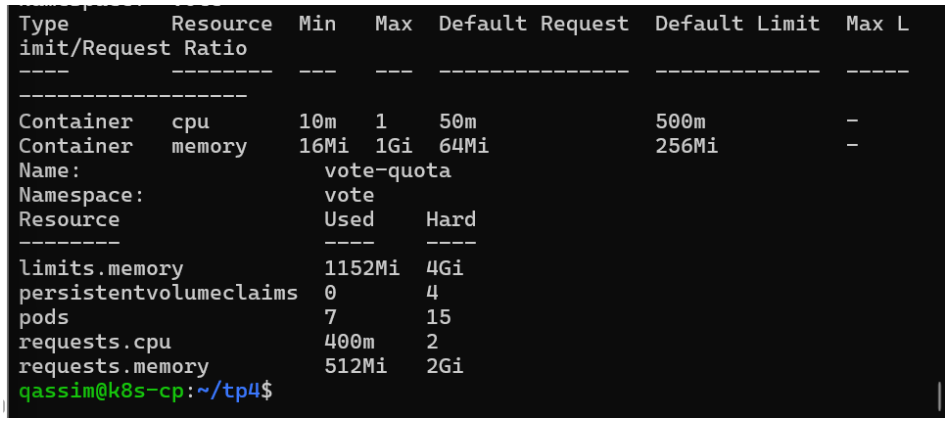


Figure 14 – Les règles par conteneur et la consommation du quota

Le **LimitRange** fixe des règles **pour chaque conteneur** : des valeurs par défaut si on n'en déclare pas, et un minimum et un maximum. Le **ResourceQuota** plafonne **le total du namespace**. Les chiffres « Utilisé » correspondent exactement à la somme de ce que j'ai déclaré : par exemple 400m de CPU demandé = 3 × 50m (vote) + 50m + 50m + 50m + 100m (db). Le quota compte donc ce qui est **déclaré**, pas ce qui est réellement consommé.

Les deux façons d'être refusé

Premier test : un pod qui demande 2 CPU, alors que le LimitRange en autorise 1 maximum.

```
qassim@k8s-cp:~/tp4$ cat > 13b-pod-refuse.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: trop-gourmand
  namespace: vote
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sleep", "3600"]
    resources:
      requests: { cpu: "2", memory: 64Mi }
      limits:   { cpu: "2", memory: 128Mi }
EOF
kubectl apply -f 13b-pod-refuse.yaml
Error from server (Forbidden): error when creating "13b-pod-refuse.yam
l": pods "trop-gourmand" is forbidden: maximum cpu usage per Container
 is 1, but limit is 2
qassim@k8s-cp:~/tp4$
```

Figure 15 – Refus immédiat par le LimitRange

Deuxième test : passer vote à 40 copies. Il restait 8 places sur les 15 pods autorisés.

```
qassim@k8s-cp:~/tp4$ kubectl -n vote scale deployment/vote --replicas=
40
sleep 20
kubectl -n vote get deploy vote
kubectl -n vote describe quota vote-quota | grep pods
kubectl -n vote describe rs -l app.kubernetes.io/name=vote | grep -m2
"exceeded quota"
kubectl -n vote scale deployment/vote --replicas=3
deployment.apps/vote scaled
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
vote   11/40   11           11          5h46m
pods                    15      15
  Warning  FailedCreate      5m50s                 replicaset-controll
er  Error creating: pods "vote-84474bc5b5-8cgsn" is forbidden: exceede
d quota: vote-quota, requested: pods=1, used: pods=15, limited: pods=1
5
  Warning  FailedCreate      5m50s                 replicaset-controll
er  Error creating: pods "vote-84474bc5b5-5xr97" is forbidden: exceede
d quota: vote-quota, requested: pods=1, used: pods=15, limited: pods=1
5
deployment.apps/vote scaled
qassim@k8s-cp:~/tp4$
```

Figure 16 – Le Deployment plafonne à 11/40 et le ReplicaSet signale le dépassement de quota

	LimitRange (pod trop gros)	ResourceQuota (trop de pods)
Ce qui est vérifié	Chaque conteneur, un par un	Le total du namespace
Quand	Tout de suite, à la création	Plus tard : la commande `scale` réussit
Où voir l'erreur	Directement dans le terminal	Dans les **événements du ReplicaSet**
Message	`maximum cpu usage per Container is 1`	`exceeded quota: vote-quota … pods=15`

Le piège est le second cas : la commande semble avoir fonctionné, et c'est en cherchant pourquoi le Deployment reste à 11/40 qu'on trouve l'explication dans `kubectl describe rs`. J'ai ensuite ramené vote à 3 copies.

Expérience : trop peu de mémoire pour la base

J'ai baissé la limite mémoire de db à 32 Mio (et sa request aussi, car une request ne peut pas dépasser la limit), puis observé le nouveau pod :

VM 1

```
sed -i ... 02-db.yaml          # requests et limits mémoire à 32Mi
kubectl apply -f 02-db.yaml
kubectl -n vote describe pod -l app.kubernetes.io/name=db | grep -A5 "Last State"
# Last State: Terminated
#   Reason:    OOMKilled
#   Exit Code: 137
```

PostgreSQL ne tient pas dans 32 Mio : le conteneur est tué dès qu'il dépasse, avec `Reason: OOMKilled` et `Exit Code: 137`, et le compteur de redémarrages augmente. J'ai ensuite remis les bonnes valeurs (128Mi / 256Mi) :

```
qassim@k8s-cp:~/tp4$ sed -i -e 's/requests: { cpu: 100m, memory: 32Mi
}/requests: { cpu: 100m, memory: 128Mi }/' \
       -e 's/limits:   { cpu: 500m, memory: 32Mi }/limits:   { cpu: 50
0m, memory: 256Mi }/' 02-db.yaml
grep -A2 "resources:" 02-db.yaml
kubectl apply -f 02-db.yaml
kubectl -n vote rollout status deployment/db
sleep 30
kubectl -n vote get pods
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits:   { cpu: 500m, memory: 256Mi }
deployment.apps/db configured
service/db unchanged
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 of 1 updated replicas
 are available...
deployment "db" successfully rolled out
NAME                      READY   STATUS    RESTARTS      AGE
db-797fddbb7c-z8mb4       1/1     Running   0             38s
redis-f8f7f7d4c-v6s69     1/1     Running   0             42m
result-6db8b56bc-9llj5    1/1     Running   1 (93s ago)   20m
vote-84474bc5b5-62dr2     1/1     Running   0             40m
vote-84474bc5b5-b4mkg     1/1     Running   0             40m
vote-84474bc5b5-gz7ps     1/1     Running   0             42m
worker-764ffdfbf9-92ckg   1/1     Running   1 (93s ago)   42m
qassim@k8s-cp:~/tp4$
```

Figure 17 – Retour aux bonnes valeurs : db de nouveau en marche

Question 4 – Dépasser une limite CPU ou une limite mémoire, quelle différence ?

• **CPU** : le conteneur n'est jamais arrêté. Le noyau Linux le **ralentit** (on parle de *throttling*) : il reçoit moins de temps de calcul et devient lent.

• **Mémoire** : c'est une frontière stricte. S'il la dépasse, le noyau le **tue** (*OOM killer*) : `OOMKilled`, code 137 (128 + 9, le signal SIGKILL). Le kubelet le redémarre, et il finit en `CrashLoopBackOff` si le problème se répète.

En résumé : trop peu de CPU rend l'application **lente**, trop peu de mémoire la fait **planter**.

Partie 4 – StorageClass et PVC

Pour garder les données de la base, il faut un vrai disque, indépendant du pod. En Kubernetes, on ne crée pas le disque soi-même : on fait une **demande** (un **PVC**, PersistentVolumeClaim), et une **StorageClass** se charge de créer le disque (le **PV**, PersistentVolume) qui y répond.

Sans StorageClass

Mon cluster kubeadm n'avait aucune StorageClass (`kubectl get sc` ne renvoyait rien). Un PVC de test de 2 Gio restait alors bloqué en `Pending`, avec le message « no persistent volumes available for this claim and no storage class is set » : personne ne pouvait créer le disque. Je l'ai supprimé.

Installer local-path-provisioner

VM 1

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-
provisioner/v0.0.31/deploy/local-path-storage.yaml
kubectl -n local-path-storage wait --for=condition=available deployment/local-
path-provisioner --timeout=120s
kubectl patch storageclass local-path -p '{"metadata": {"annotations":
{"storageclass.kubernetes.io/is-default-class":"true"}}}'
kubectl get storageclass
```

local-path crée les disques sous forme de simples dossiers sur le nœud. Je l'ai déclarée comme classe **par défaut**. Elle fonctionne en mode `WaitForFirstConsumer`, ce qui explique la suite.

La base avec un PVC

14a-db-deployment-pvc.yaml (extrait)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
  namespace: vote
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path
  resources:
    requests:
      storage: 2Gi
---
# Dans le Deployment db :
        env:
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: db-data
```

`PGDATA` pointe vers un sous-dossier, car le provisioner peut déposer des fichiers à la racine du disque, et PostgreSQL refuse de créer une base dans un dossier qui n'est pas vide.

J'ai d'abord créé le PVC **seul**, sans pod pour l'utiliser :

```
qassim@k8s-cp:~/tp4$ kubectl create -f <(sed -n '1,/^---$/p' 14a-db-de
ployment-pvc.yaml | sed '$d')
sleep 10
kubectl -n vote get pvc db-data
kubectl -n vote describe pvc db-data | tail -3
persistentvolumeclaim/db-data created
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS   AGE
db-data   Pending                                      local-path
<unset>                 10s
  Type    Reason                Age               From
        Message
  ----    ------                ----              ----
        -------
  Normal  WaitForFirstConsumer  6s (x2 over 10s)  persistentvolume-con
troller  waiting for first consumer to be created before binding
```

Figure 18 – Sans pod, le PVC reste Pending : il attend son premier utilisateur

Puis j'ai appliqué le Deployment :

```
qassim@k8s-cp:~/tp4$ kubectl apply -f 14a-db-deployment-pvc.yaml
kubectl -n vote rollout status deployment/db
kubectl -n vote get pvc,pv
kubectl -n vote get pods -l app.kubernetes.io/name=db -o wide
kubectl get pv -o custom-columns=NOM:.metadata.name,CHEMIN:.spec.hostP
ath.path,CHEMIN_LOCAL:.spec.local.path,NOEUD:.spec.nodeAffinity.requir
ed.nodeSelectorTerms[0].matchExpressions[0].values[0]
Warning: resource persistentvolumeclaims/db-data is missing the kubect
l.kubernetes.io/last-applied-configuration annotation which is require
d by kubectl apply. kubectl apply should only be used on resources cre
ated declaratively by either kubectl create --save-config or kubectl a
pply. The missing annotation will be patched automatically.
persistentvolumeclaim/db-data configured
deployment.apps/db configured
service/db unchanged
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 of 1 updated replicas
 are available...
deployment "db" successfully rolled out
NAME                           STATUS   VOLUME
            CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
SCLASS   AGE
persistentvolumeclaim/db-data   Bound    pvc-ba79d56a-61f8-4f7c-a95e-5
74b0cdeabc8   2Gi        RWO            local-path     <unset>
         35s

NAME                                                        CAPACITY
 ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGECLASS
  VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-ba79d56a-61f8-4f7c-a95e-574b0cdeabc8   2Gi
 RWO            Delete           Bound    vote/db-data   local-path
```

Figure 19 – Dès que le pod démarre, le PVC passe Bound et un PV est créé

```
service/db unchanged
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 out of 1 new replicas
 have been updated...
Waiting for deployment "db" rollout to finish: 0 of 1 updated replicas
 are available...
deployment "db" successfully rolled out
NAME                              STATUS   VOLUME
            CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
SCLASS   AGE
persistentvolumeclaim/db-data   Bound    pvc-ba79d56a-61f8-4f7c-a95e-5
74b0cdeabc8   2Gi        RWO            local-path     <unset>
         35s

NAME                                                        CAPACITY
 ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGECLASS
  VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-ba79d56a-61f8-4f7c-a95e-574b0cdeabc8   2Gi
 RWO            Delete           Bound    vote/db-data   local-path
  <unset>                          9s
NAME                 READY   STATUS    RESTARTS   AGE   IP
  NODE     NOMINATED NODE   READINESS GATES
db-588467454-t4cwc   1/1     Running   0          16s   10.244.228.97
  k8s-w1   <none>           <none>
NOM                                          CHEMIN
                                                        CHEMIN_LOCAL
  NOEUD
pvc-ba79d56a-61f8-4f7c-a95e-574b0cdeabc8   /opt/local-path-provisioner
/pvc-ba79d56a-61f8-4f7c-a95e-574b0cdeabc8_vote_db-data   <none>
  k8s-w1
qassim@k8s-cp:~/tp4$
```

Figure 20 – Le volume est créé sur k8s-w1, dans /opt/local-path-provisioner

Le PVC est passé de **Pending** à **Bound**, et un PV de 2 Gio a été créé automatiquement. Le pod db tourne sur **k8s-w1**, et le volume a été créé **sur ce même nœud**, dans `/opt/local-path-provisioner/pvc-ba79d56a-…_vote_db-data`. Je suis allé le voir directement sur le disque du worker :

```
qassim@k8s-w1:~$ sudo ls -l /opt/local-path-provisioner/
sudo ls /opt/local-path-provisioner/*db-data*/
sudo ls /opt/local-path-provisioner/*db-data*/pgdata | head
[sudo: authenticate] Password:
total 4
drwxrwxrwx 3 root root 4096 Oct 27 14:13 pvc-ba79d56a-61f8-4f7c-a95e-574b0cd
eabc8_vote_db-data
pgdata
PG_VERSION
base
global
pg_commit_ts
pg_dynshmem
pg_hba.conf
pg_ident.conf
pg_logical
pg_multixact
pg_notify
qassim@k8s-w1:~$
```

Figure 21 – Sur k8s-w1 : le dossier du volume contient les fichiers de PostgreSQL

Le volume n'est qu'un dossier sur le disque du nœud, monté dans le conteneur. On y retrouve le sous-dossier `pgdata` et les fichiers de PostgreSQL (`base`, `global`, `PG_VERSION`…). Il survit au pod, mais pas à la perte du nœud : c'est parfait pour un TP, pas pour une production qui a besoin de haute disponibilité.

> **Question 5 – Pourquoi le PVC reste Pending tant qu'aucun pod ne l'utilise ?** À cause du champ **`volumeBindingMode: WaitForFirstConsumer`** de la StorageClass. Un disque local n'existe que sur **un**

> **seul nœud**. S'il était créé tout de suite, il pourrait l'être sur un nœud où le pod ne pourra jamais aller. Le provisioner **attend donc que le scheduler ait choisi le nœud du pod**, puis crée le disque sur ce nœud. Ce Pending n'est pas une erreur, c'est une attente volontaire.

Partie 5 – Passage en StatefulSet

Un **StatefulSet** est fait pour les applications qui gardent des données, comme les bases. Contrairement à un Deployment, il donne à chaque pod un **nom fixe** (`db-0`) et lui associe **son propre disque**, toujours le même.

La migration

J'ai d'abord voté pour avoir des données (2 votes), puis supprimé le Deployment db, **sans supprimer le PVC** :

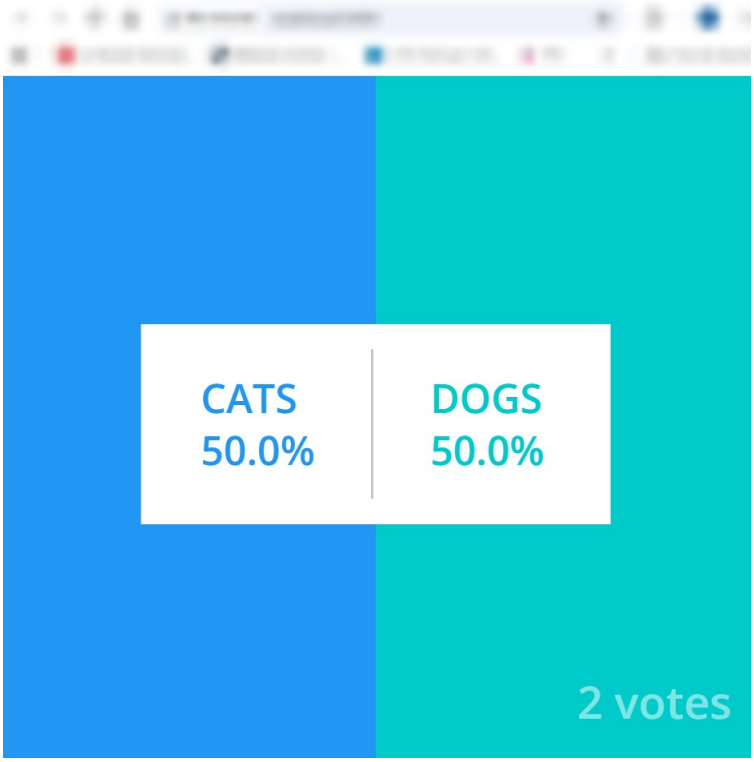


Figure 22 – Avant la migration : 2 votes

VM 1

```
kubectl -n vote delete deployment db      # le PVC db-data reste
```

Le nouveau fichier contient trois objets : le Service `db` (inchangé), un Service **headless** `db-hl` (sans IP virtuelle) et le StatefulSet :

14-db-statefulset.yaml (extrait)

```
apiVersion: v1
kind: Service
metadata:
  name: db-hl
  namespace: vote
spec:
  clusterIP: None          # Service headless
  selector:
    app.kubernetes.io/name: db
  ports:
  - name: postgres
    port: 5432
```

```
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
  namespace: vote
spec:
  serviceName: db-hl
  replicas: 1
  template:
    # conteneur identique : Secret, PGDATA, sondes, ressources
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: local-path
      resources:
        requests:
          storage: 2Gi
```

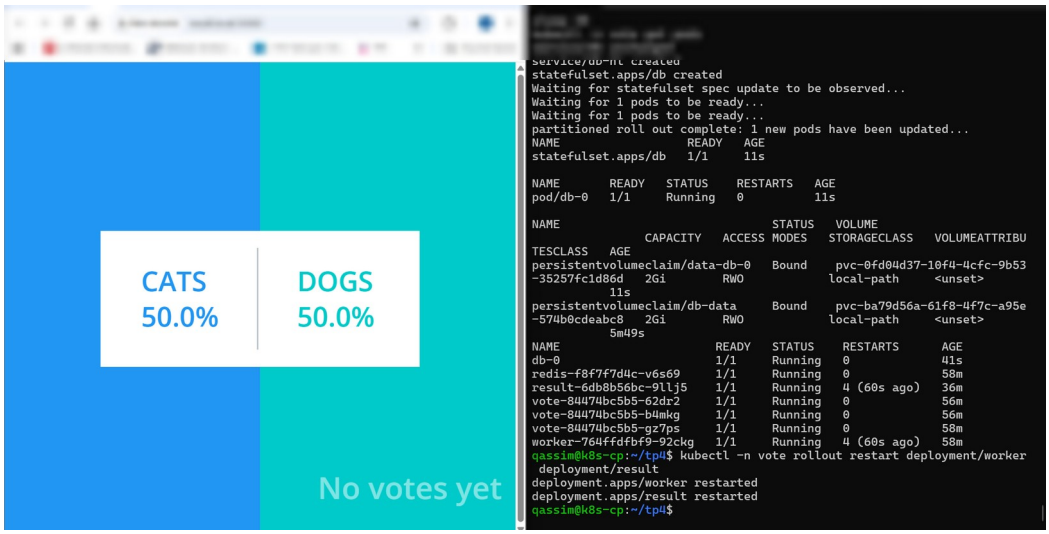


Figure 23 – Le pod db-0 et le PVC data-db-0, mais plus aucun vote

Le pod s'appelle **`db-0`** et le PVC **`data-db-0`** (modèle : `<nom du template>-<nom du StatefulSet>-<numéro>`). Les 2 votes d'avant ont **disparu** : le StatefulSet n'a pas repris le PVC `db-data`. Il a créé son propre PVC, vide, à partir de son modèle. Pour garder les données, il aurait fallu nommer le PVC existant `data-db-0` dès le départ (un StatefulSet adopte un PVC qui porte déjà le bon nom), ou copier les données d'un disque à l'autre. J'ai ensuite supprimé l'ancien PVC devenu inutile.

Test de persistance

J'ai voté à nouveau (1 vote), puis soumis la base à plusieurs épreuves.

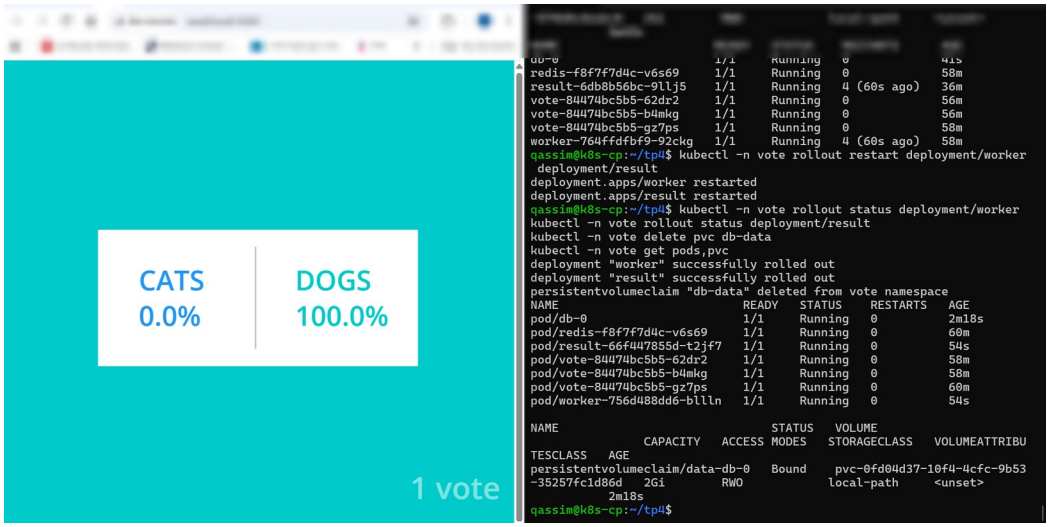


Figure 24 – Ancien PVC supprimé, un nouveau vote enregistré

Épreuve 1 : supprimer le pod db-0.

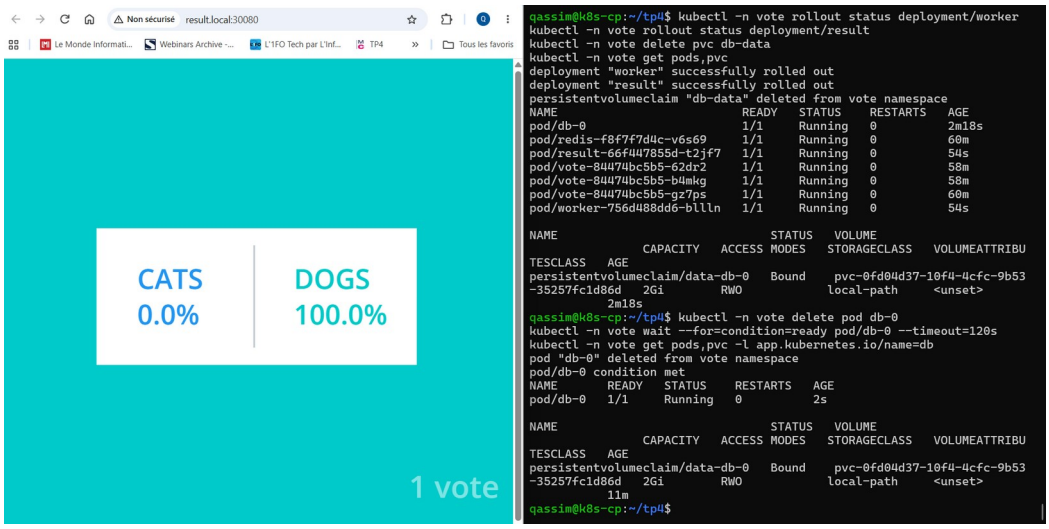


Figure 25 – Un nouveau pod db-0 relié au même PVC, le vote est toujours là

Kubernetes a recréé un pod **sous le même nom**, `db-0`, relié au **même** PVC. Le vote est toujours là.

Épreuve 2 : supprimer le StatefulSet entier, puis le recréer.

VM 1

```
kubectl -n vote delete sts db
kubectl -n vote get pvc            # data-db-0 toujours Bound
kubectl apply -f 14-db-statefulset.yaml
```

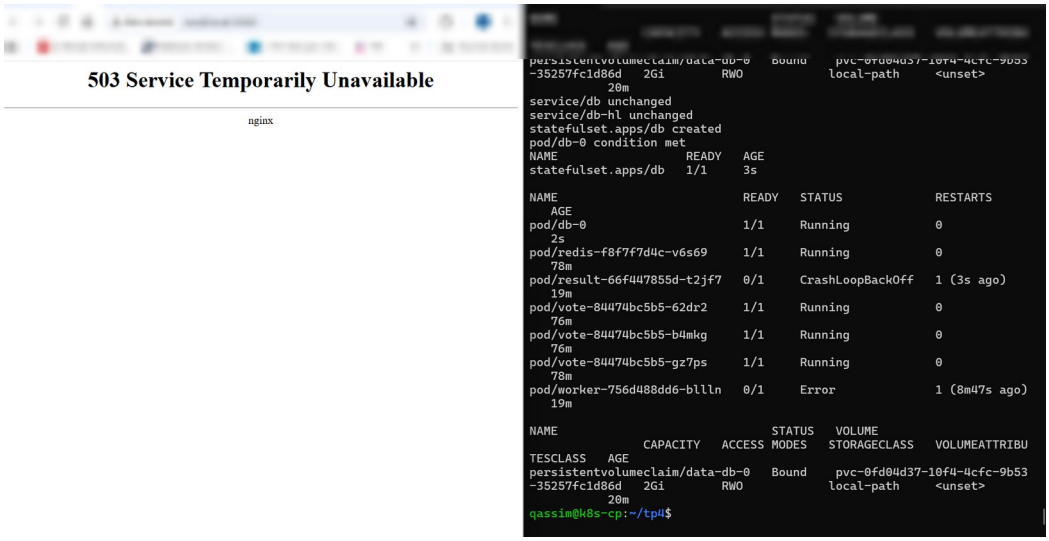


Figure 26 – Le PVC survit, mais result affiche une erreur 503 pendant quelques secondes

Le PVC `data-db-0` est resté **Bound** pendant toute l'opération, et le StatefulSet recréé l'a repris. La page result a d'abord affiché une **erreur 503** : result venait de perdre sa base, sa readiness échouait, et Kubernetes l'avait donc retiré du Service. L'Ingress n'avait plus aucun pod vers qui envoyer la requête. C'est la readiness qui protège le trafic. Une minute plus tard, tout était revenu :

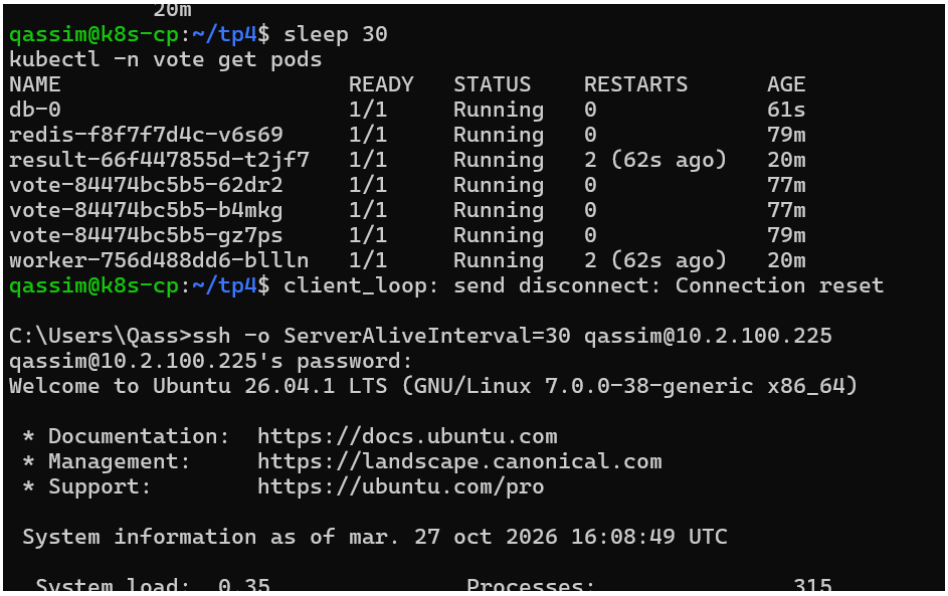


Figure 27 – Tous les pods sont revenus seuls

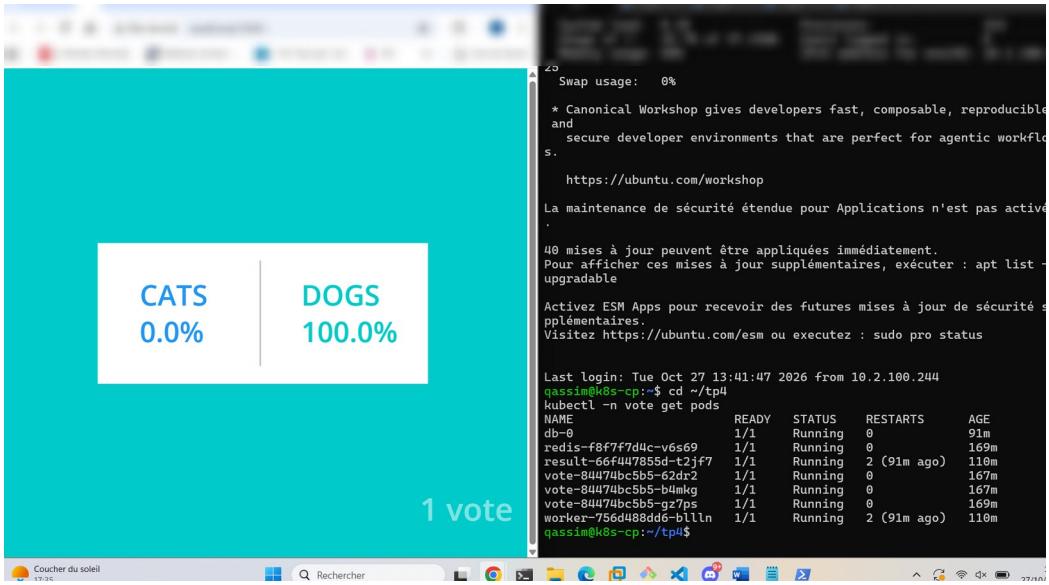


Figure 28 – Le vote a survécu à la suppression du StatefulSet

Ce qu'on supprime	TP3 (emptyDir)	TP4 (StatefulSet + PVC)
Le pod de la base	Votes **perdus**	Votes **conservés**
Le StatefulSet entier	—	Votes **conservés** (le PVC survit)
Éviction par un drain (Partie 6)	—	Votes **conservés**

Les PVC créés par `volumeClaimTemplates` survivent à la suppression du StatefulSet : c'est le comportement par défaut, pour ne jamais perdre de données par erreur.

Résolution DNS

J'ai lancé un pod temporaire pour interroger le DNS du cluster. Mon premier essai avec des noms courts a échoué sur `db-0.db-hl` (`NXDOMAIN`) : l'outil `nslookup` de busybox gère mal les noms courts qui contiennent un point. Avec les noms complets, tout fonctionne :

VM 1

```
kubectl -n vote run dns --image=busybox:1.36 --restart=Never --rm -it -- sh -c \
  'nslookup db.vote.svc.cluster.local; nslookup db-0.db-
hl.vote.svc.cluster.local; nslookup db-hl.vote.svc.cluster.local'
```

```
qassim@k8s-cp:~/tp4$ kubectl -n vote run dns --image=busybox:1.36 --re
start=Never --rm -it -- sh -c 'nslookup db.vote.svc.cluster.local; ech
o ------; nslookup db-0.db-hl.vote.svc.cluster.local; echo ------; nsl
ookup db-hl.vote.svc.cluster.local'
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   db.vote.svc.cluster.local
Address: 10.103.193.148


------
Server:         10.96.0.10
Address:        10.96.0.10:53


Name:   db-0.db-hl.vote.svc.cluster.local
Address: 10.244.228.76

------
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   db-hl.vote.svc.cluster.local
Address: 10.244.228.76


pod "dns" deleted from vote namespace
qassim@k8s-cp:~/tp4$
```

Figure 29 – db renvoie l'IP du Service, db-0.db-hl l'IP du pod

Nom	Réponse	Ce que c'est
`db`	10.103.193.148	L'IP virtuelle du Service ; kube-proxy choisit un pod derrière
`db-0.db-hl`	10.244.228.76	L'IP **du pod db-0 lui-même**
`db-hl`	10.244.228.76	Un Service headless renvoie directement les IP des pods

> **Question 6 – À quoi sert db-hl si l'application n'utilise que db ?** Avec une seule copie, `db-hl` sert surtout à remplir le champ obligatoire `serviceName` du StatefulSet, et à donner au pod un **nom DNS stable et individuel**, `db-0.db-hl`. Ce nom devient **indispensable** dès qu'il y a plusieurs copies avec des rôles différents. Par exemple, avec une réplication PostgreSQL (un primaire et des secondaires), les secondaires doivent se connecter précisément à `db-0`, le primaire, et pas à n'importe quel pod derrière `db`. Les bases en cluster (PostgreSQL, MongoDB, Kafka…) reposent toutes sur ce mécanisme.

Partie 6 – PodDisruptionBudget

Un **PodDisruptionBudget** (PDB) fixe un nombre minimum de pods à garder disponibles pendant les opérations volontaires, comme la maintenance d'un nœud.

16-pdb-vote.yaml

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: vote
  namespace: vote
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app.kubernetes.io/name: vote
```

Avec 3 pods vote et un minimum de 2, on peut en retirer **1** à la fois (`ALLOWED DISRUPTIONS 1`). J'ai ensuite lancé la vidange (drain) du worker :

VM 1

```
kubectl drain k8s-w1 --ignore-daemonsets --delete-emptydir-data
```

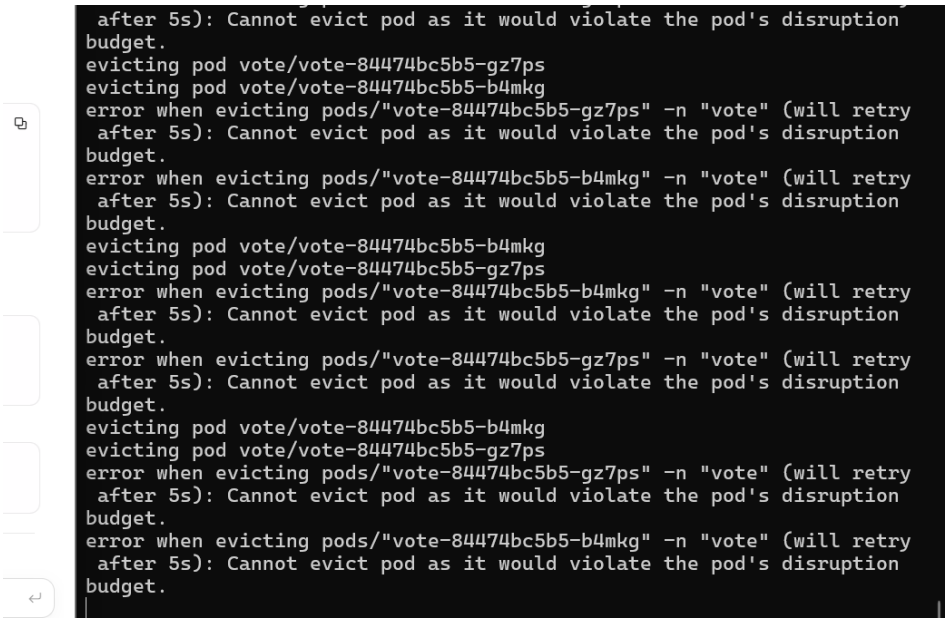


Figure 30 – Le drain est bloqué par le PDB et réessaie toutes les 5 secondes

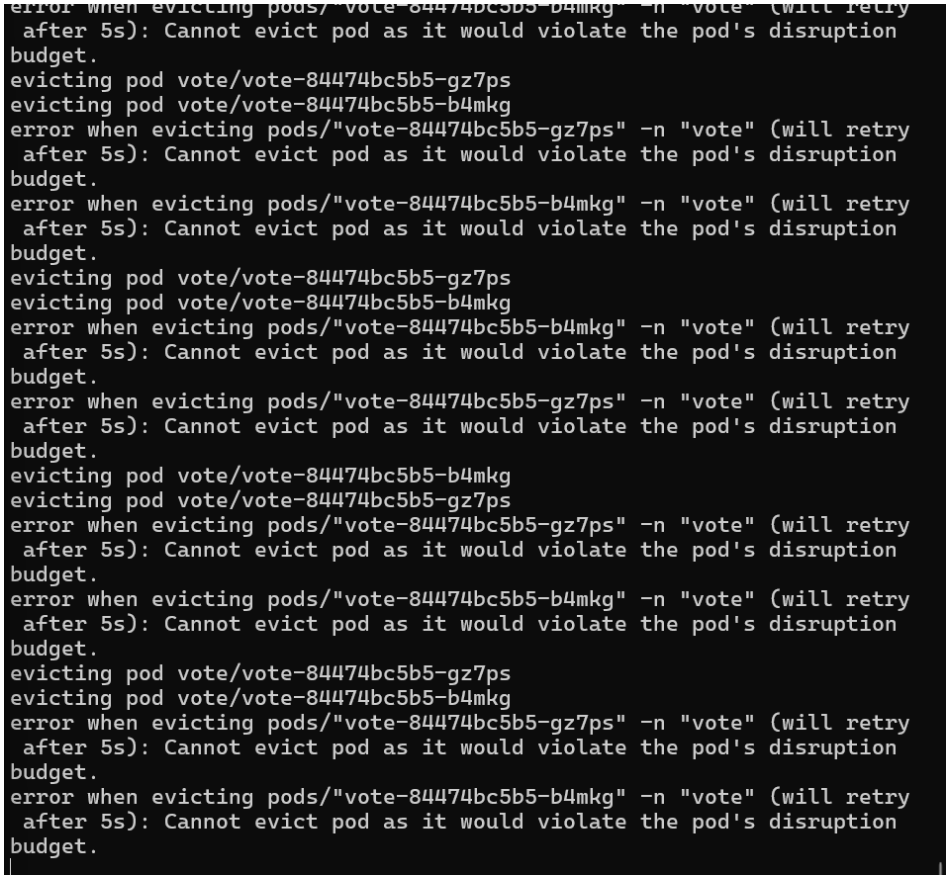


Figure 31 – Le drain réessaie sans fin

Le drain a retiré un premier pod vote, puis s'est **bloqué** : il réessaie toutes les 5 secondes avec `Cannot evict pod as it would violate the pod's disruption budget`. Je l'ai arrêté avec Ctrl+C et j'ai remis le nœud en service :

```
qassim@k8s-cp:~/tp4$ kubectl -n vote get pdb
kubectl -n vote get pods -o wide
kubectl uncordon k8s-w1
kubectl get nodes
NAME   MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
vote   2               N/A               0                     2m10s
NAME                      READY   STATUS    RESTARTS   AGE    IP
         NODE     NOMINATED NODE   READINESS GATES
db-0                      0/1     Pending   0          2m2s   <none>
         <none>   <none>           <none>
redis-f8f7f7d4c-6qqnn     0/1     Pending   0          2m3s   <none>
         <none>   <none>           <none>
result-66f447855d-5fc8c   0/1     Pending   0          2m3s   <none>
         <none>   <none>           <none>
vote-84474bc5b5-b4mkg     1/1     Running   0          170m   10.244.2
28.126   k8s-w1   <none>           <none>
vote-84474bc5b5-gz7ps     1/1     Running   0          172m   10.244.2
28.123   k8s-w1   <none>           <none>
vote-84474bc5b5-z5h47     0/1     Pending   0          2m3s   <none>
         <none>   <none>           <none>
worker-756d488dd6-trllv   0/1     Pending   0          2m3s   <none>
         <none>   <none>           <none>
node/k8s-w1 uncordoned
NAME     STATUS   ROLES           AGE   VERSION
k8s-cp   Ready    control-plane   27h   v1.34.12
k8s-w1   Ready    worker          27h   v1.34.12
qassim@k8s-cp:~/tp4$ sleep 60
kubectl -n vote get pods
```

Figure 32 – Budget épuisé, pods en attente, puis nœud remis en service

On voit que :

- le PDB est à **`ALLOWED DISRUPTIONS 0`** : on ne peut plus retirer aucun pod vote ;
- les **deux pods vote protégés** sont restés en marche, et le troisième attend en `Pending`, car il ne peut aller nulle part ;
- les autres composants (db-0, redis, result, worker) n'ont **pas de PDB** : ils ont été retirés sans résistance et attendaient aussi en `Pending`.

Après `kubectl uncordon`, tout a redémarré sur le worker en une minute, et le vote était toujours là :

```
qassim@k8s-cp:~/tp4$ sleep 60
kubectl -n vote get pods
NAME                      READY   STATUS    RESTARTS   AGE
db-0                      1/1     Running   0          3m10s
redis-f8f7f7d4c-6qqnn     1/1     Running   0          3m11s
result-66f447855d-5fc8c   1/1     Running   0          3m11s
vote-84474bc5b5-b4mkg     1/1     Running   0          171m
vote-84474bc5b5-gz7ps     1/1     Running   0          173m
vote-84474bc5b5-z5h47     1/1     Running   0          3m11s
worker-756d488dd6-trllv   1/1     Running   0          3m11s
qassim@k8s-cp:~/tp4$
```

Figure 33 – Les 7 pods de retour, les 2 pods vote protégés n'ont jamais été interrompus

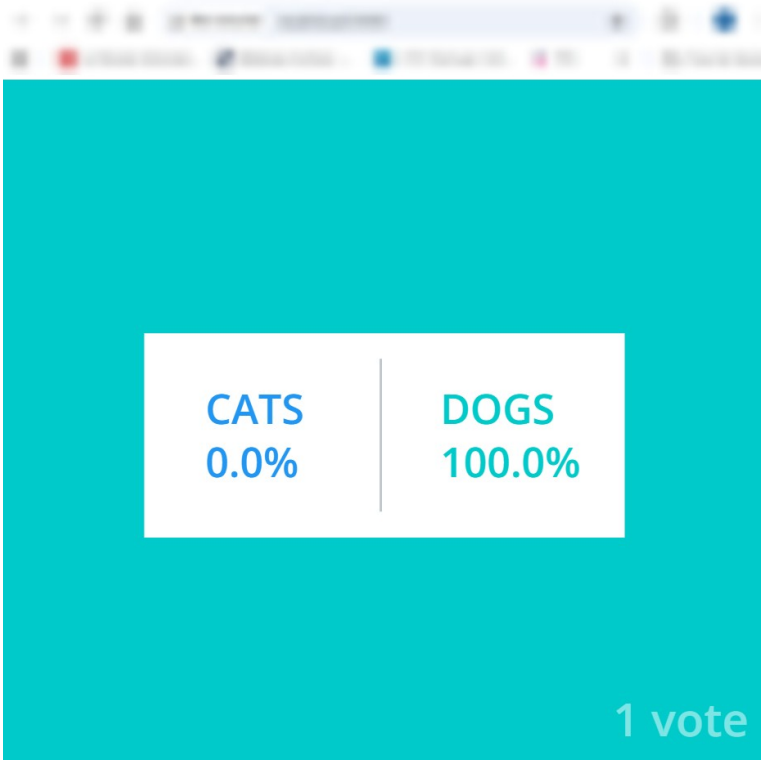


Figure 34 – Le vote a même survécu à l'éviction de la base par le drain

> **Question 7 – Le PDB empêche-t-il le drain, ou le ralentit-il seulement ?**
>
> • **Avec un seul worker**, il le **bloque** : le pod retiré ne peut être recréé nulle part (le worker est en cours de vidange, le control plane refuse les pods applicatifs). On ne remonte jamais à 3 pods, et retirer les deux autres violerait le budget. La commande réessaie sans fin.
>
> • **Avec deux workers**, il le **ralentit** seulement : chaque pod retiré est recréé sur l'autre worker, le nombre remonte à 3, et le drain peut retirer le suivant. Les pods sont déplacés **un par un**, sans jamais descendre sous 2. C'est exactement ce qu'on veut lors d'une mise à jour de nœuds.

Cette expérience montre aussi qu'un PDB ne protège que les pods qu'il sélectionne. En production, on en mettrait un sur chaque composant important. Pour une base à une seule copie, un PDB bloquerait toute maintenance : il faudrait d'abord plusieurs copies.

Partie 7 – Pour aller plus loin (non réalisée)

Je n'ai pas réalisé les bonus. Voici ce qu'ils auraient apporté :

- **Redis persistant** : un PVC de 1 Gio monté sur `/data` et l'option `--appendonly yes`. Dans cette application, redis n'est qu'une file d'attente très courte : on ne perdrait que les votes pas encore traités par le worker, soit quelques millisecondes. La persistance ne vaut le coup que si un vote perdu coûte cher.
- **Sauvegarde logique** : un CronJob quotidien qui lance `pg_dump` vers un PVC de sauvegarde et garde 7 fichiers, puis une restauration avec `psql` après avoir simulé une perte. Leçon importante : **un PVC n'est pas une sauvegarde**. Si le nœud ou le disque disparaît, les données partent avec. Un vrai dump, copié hors du cluster (stockage S3, NAS), en est une.

Livrables et état final

VM 1

```
kubectl -n vote get pvc,pv,sts,pdb
```

```
** server can't find db-0.db-hl: NXDOMAIN

** server can't find db-0.db-hl: NXDOMAIN

pod "dns" deleted from vote namespace
pod vote/dns terminated (Error)
NAME    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
db      ClusterIP   10.103.193.148   <none>        5432/TCP   8h
db-hl   ClusterIP   None             <none>        5432/TCP   118m
NAME   READY   STATUS    RESTARTS   AGE     IP              NODE     N
OMINATED NODE   READINESS GATES
db-0   1/1     Running   0          6m18s   10.244.228.76   k8s-w1   <
none>           <none>
qassim@k8s-cp:~/tp4$ kubectl -n vote get pvc,pv,sts,pdb
NAME                                STATUS   VOLUME
                CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBU
TESCLASS   AGE
persistentvolumeclaim/data-db-0   Bound    pvc-0fd04d37-10f4-4cfc-9b53
-35257fc1d86d   2Gi        RWO            local-path     <unset>
          118m

NAME                                                        CAPACITY
 ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM           STORAGECLAS
S   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-0fd04d37-10f4-4cfc-9b53-35257fc1d86d   2Gi
 RWO            Delete           Bound    vote/data-db-0   local-path
    <unset>                          118m

NAME                  READY   AGE
statefulset.apps/db   1/1     98m

NAME                            MIN AVAILABLE   MAX UNAVAILABLE   AL
LOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/vote   2               N/A               1
                    6m40s
qassim@k8s-cp:~/tp4$
```

Figure 35 – Sortie demandée : PVC, PV, StatefulSet et PDB

Objet	État final
PVC `data-db-0`	Bound, 2 Gio, RWO, local-path
PV `pvc-0fd04d37-…`	Bound à `vote/data-db-0`, politique `Delete`
StatefulSet `db`	1/1
PDB `vote`	Minimum 2, 1 interruption autorisée (3 pods vote)

Tous les fichiers ont été copiés sur mon PC, pour être versionnés dans Git au jour 3 :

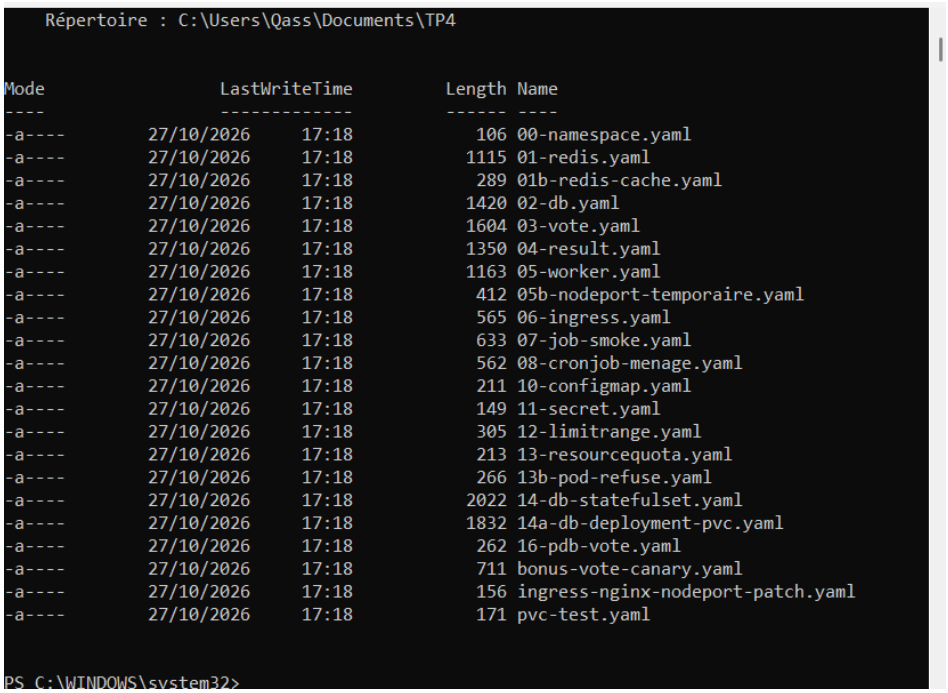


Figure 36 – Les fichiers du TP4 sur le PC

Fichier	Contenu
`01-redis.yaml` à `05-worker.yaml`	Fichiers du TP3 modifiés : ConfigMap, Secret, sondes, ressources
`10-configmap.yaml`	Options de vote
`11-secret.yaml`	Identifiants de la base
`12-limitrange.yaml` / `13-resourcequota.yaml`	Règles par conteneur et plafonds du namespace
`13b-pod-refuse.yaml`	Pod de test refusé par le LimitRange
`14a-db-deployment-pvc.yaml`	Base en Deployment avec un PVC (étape intermédiaire)
`14-db-statefulset.yaml`	Base en StatefulSet, Services db et db-hl
`16-pdb-vote.yaml`	PodDisruptionBudget de vote

> **Attention avant de versionner :** `11-secret.yaml` contient le mot de passe, seulement encodé en base64. On ne met jamais un Secret en clair dans Git : on utilise des outils comme Sealed Secrets ou SOPS, qui le chiffrent. Ici, le mot de passe est imposé par l'application, mais la règle reste valable.

Difficultés rencontrées

Problème	Cause	Solution
Archive TP4.zip illisible	Fichier abîmé par la plateforme	Fichiers écrits moi-même
Worker et result en erreur après chaque changement de db	Ils perdent leur connexion quand la base est recréée	Ils reviennent seuls ; sinon `rollout restart`
Fichiers créés dans le mauvais dossier	Reconnexion SSH dans `~`	Déplacement dans `~/tp4`, puis `cd ~/tp4` à chaque reconnexion
Erreur 503 sur result	Pod retiré du Service par sa readiness pendant la reconnexion	Attendre une minute
Votes perdus lors du passage en StatefulSet	Le StatefulSet crée son propre PVC	Comportement attendu ; il aurait fallu nommer le PVC `data-db-0`
`NXDOMAIN` sur `db-0.db-hl`	Limite de `nslookup` dans busybox	Utiliser les noms DNS complets
Drain bloqué	Le PDB empêche de descendre sous 2 pods	Comportement attendu ; Ctrl+C puis `uncordon`

Conclusion

Ce TP a transformé une application qui « marche » en une application beaucoup plus solide. La configuration est maintenant séparée du code, dans une ConfigMap et un Secret. J'ai compris qu'une variable d'environnement n'est lue qu'au démarrage, et qu'il faut donc relancer les pods pour qu'un changement de configuration soit pris en compte.

Les sondes m'ont le plus appris. La readiness rend les mises à jour plus propres, car aucun visiteur n'est envoyé vers un pod pas encore prêt, et elle a même protégé le trafic pendant les coupures de la base (erreur 503 plutôt qu'une page cassée). La liveness, elle, est puissante mais dangereuse : mal réglée, elle fait redémarrer en boucle une application qui fonctionne.

Les ressources rendent le cluster prévisible et équitable : chaque conteneur réserve ce dont il a besoin, et le namespace ne peut pas dépasser son budget. Retenir la différence entre CPU et mémoire est essentiel : trop peu de CPU ralentit, trop peu de mémoire tue.

Enfin, le passage en StatefulSet avec un volume persistant a réglé le problème du TP3 : les votes ont survécu à la suppression du pod, à la suppression du StatefulSet et à la maintenance du nœud. Il reste une limite importante : un volume persistant n'est pas une sauvegarde. La prochaine étape logique serait une vraie sauvegarde, copiée hors du cluster, puis la gestion de tous ces fichiers dans Git avec le GitOps du jour 3.